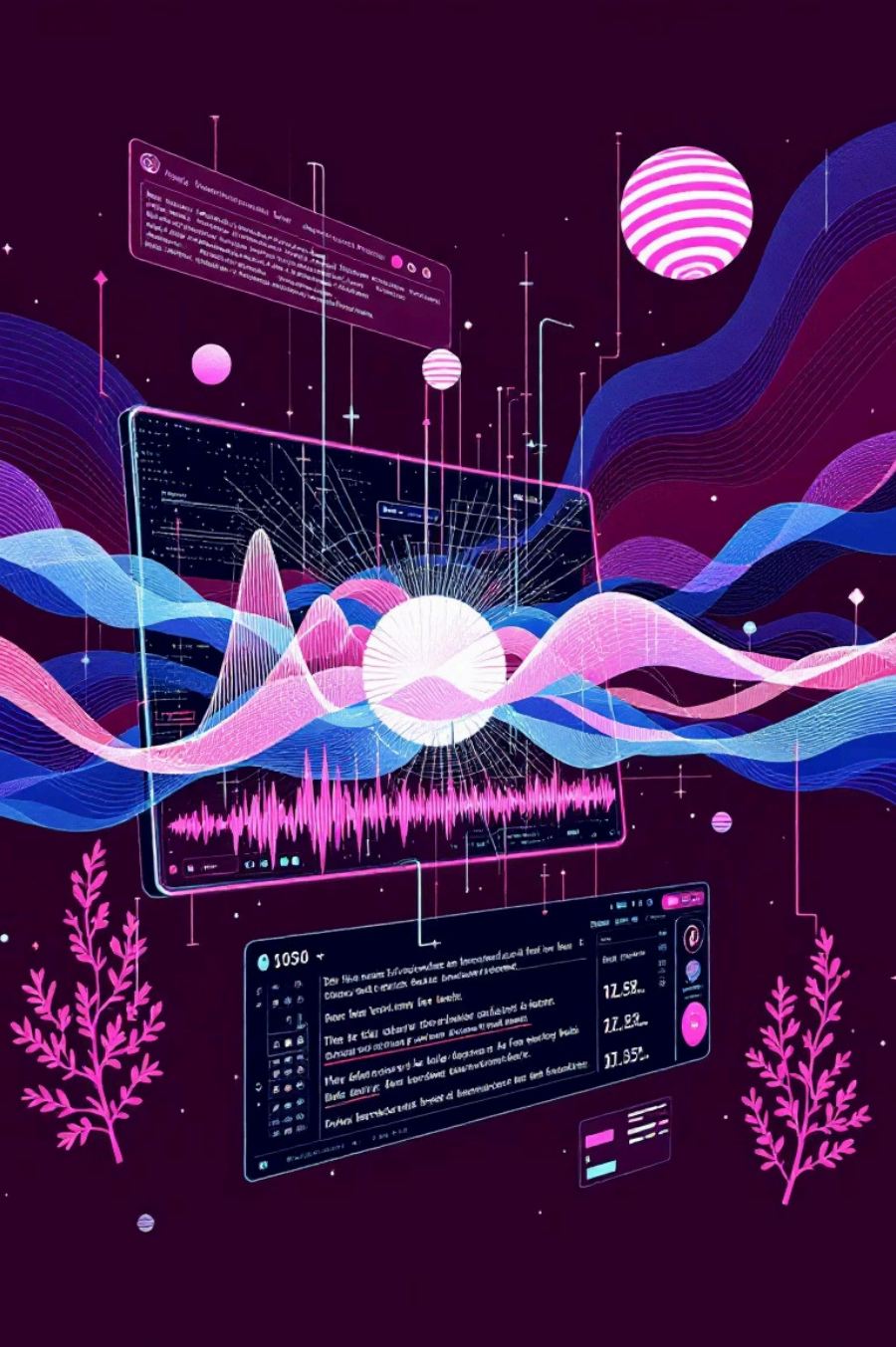




Speech-to-Text AI: From Voice to Text with Intelligent Algorithms

Team name

Abdallah Mohamed Fathy Abdel-Aaty

Abdulrahman Mohamed Ali Ahmed

Amr Khaled Ali Mohamed

Abdallah Ali Nabeh Abdel-Galil

Mahmoud Ahmed Mahmoud Emam

Youssef Radwan Ibrahim Mbarek

Mohamed Ahmed Saad Ali

Salma Mohamed Abdelaziz Mohamed

Salma Walid Sami Fathy

Maha Romany Karimi Falihi

What is Speech-to-Text AI?

Speech-to-Text (STT) AI is a sophisticated technology that **transforms spoken language into written text** using advanced artificial intelligence and machine learning algorithms.

- It enables hands-free transcription, powering **virtual assistants**, **accessibility tools**, and various innovative applications.
- STT is the core technology underpinning popular services like Siri, Alexa, and numerous real-time transcription applications, making digital interactions more **intuitive and inclusive**.



Key Components of Speech-to-Text Systems

1

Audio Processing

Initial steps involve noise removal, volume normalization, and segmenting the audio signal for clearer analysis.

2

Feature Extraction

Audio is converted into meaningful numerical features, such as Mel-Frequency Cepstral Coefficients (MFCCs), ready for model input.

3

Acoustic Modeling

Deep neural networks are employed to map the extracted audio features to phonemes or sub-word units, identifying the basic sounds.

4

Language Modeling

This component predicts the most probable sequence of words, ensuring grammatical correctness and contextual relevance.

5

Decoding

Finally, the outputs from both the acoustic and language models are combined to generate the coherent and accurate final text output.

Algorithms in Text-to-Speech Studio

Transformer Models

Purpose: Foundation of modern TTS systems

Function: Uses self-attention mechanisms to understand context and generate natural speech with proper prosody and intonation

Used in: Core architecture of XTTS v2 model for both text understanding and speech generation

XTTS v2 (Cross-lingual Text-to-Speech)

Purpose: Advanced multilingual TTS model based on Transformer architecture

Function: Leverages Transformer-based encoder-decoder architecture with attention mechanisms for high-quality voice synthesis

Used in: Main voice generation engine with speaker adaptation

Voice Cloning / Speaker Adaptation

Purpose: Adapt generated speech to match a reference voice

Function: Analyzes speaker characteristics from voice samples and applies them to synthesized speech

Used in: Creating personalized voice output

Text Segmentation Algorithm

Purpose: Split long text into manageable chunks

Function: Divides text naturally at sentence boundaries (., !, ?, etc.) while respecting max length constraints

Used in: Processing long documents into parts

Flowchart: Speech-to-Text Process Overview

```
graph LR; A[Audio Input] --> B[Preprocessing]; B --> C[Feature Extraction]; C --> D[Acoustic Model];
```

Audio Input

Preprocessing

Feature Extraction

Acoustic Model

Pseudocode for Speech-to-Text Algorithm

```
FUNCTION SpeechToText(audio_input):
    // Step 1: Audio Input & Preprocessing
    audio_processed = PreprocessAudio(audio_input)

    // Step 2: Feature Extraction
    audio_features = ExtractFeatures(audio_processed)

    // Step 3: Acoustic Modeling
    phoneme_probabilities = AcousticModel(audio_features)

    // Step 4: Language Modeling
    word_sequence_probabilities = LanguageModel(phoneme_probabilities)

    // Step 5: Decoding
    final_text = Decode(phoneme_probabilities, word_sequence_probabilities)

    RETURN final_text

FUNCTION PreprocessAudio(raw_audio):
    // Apply noise reduction, volume normalisation, sampling, and segmentation
    processed_audio = ApplyFilters(raw_audio)
    RETURN processed_audio

FUNCTION ExtractFeatures(processed_audio):
    // Compute MFCCs or other relevant features
    features = ComputeMFCC(processed_audio)
    RETURN features

FUNCTION AcousticModel(features):
    // Use a trained deep neural network (e.g., CNN, RNN) to map features to phonemes
    probabilities = DNN_Predict(features)
    RETURN probabilities

FUNCTION LanguageModel(phoneme_probs):
    // Use a statistical or neural language model to predict word sequences
    word_sequences = NGRAM_OR_TRANSFORMER_Predict(phoneme_probs)
    RETURN word_sequences

FUNCTION Decode(acoustic_probs, language_probs):
    // Combine probabilities to find the most likely word sequence
    decoded_text = ViterbiSearch(acoustic_probs, language_probs)
    RETURN decoded_text
```